Hardened Flask REST API Production Documentation & Engineering Guide

---

1. The Waiter Analogy: How This Engine Actually Works

If you've never built an **API (Application Programming Interface)** before, don't sweat it. Think of this Python script (`main.py`) as a **highly trained, no-nonsense waiter in a secure restaurant**.

- **The Customer:** Your local WordPress frontend form.
- **The Waiter:** This Flask REST API software.
- **The Kitchen Vault:** Your private, locked-down MySQL Database on Azure.

Your local WordPress site can't talk directly to the database on your Azure server—that would be a massive security hazard. If a random bot or hacker could touch your data directly, they could wipe it clean.

Instead, when a visitor types their information into your form and hits submit, your WordPress site flags down the **Waiter (Our Flask API)**.

text

```
[ Local WP Frontend ] ──(1. Form Submitted)──> [ Waiter (Flask REST API on Port 8000) ]
                                                                  |
                                                  (2. Bouncer Checks Secret Key Stamp)
                                                                  |
                                                                  ▼
                                                  (3. Snipps and Enriches Data Logic)
                                                                  |
                                                                  ▼
                                                [ Kitchen Vault (MySQL Database on 127.0.0.1) ]
```

Use code with caution.

The Waiter catches the envelope, checks the signature header to ensure it's from a verified friend, rips it open, strips and grades the email data, runs to the back to lock it inside the **Kitchen Vault (MySQL)**, and runs back out to give your website a happy success receipt.

---

2. Production Inbound Delivery Guidelines

To successfully flag down the Waiter, your web requests must adhere to these strict delivery rules, or the server will drop the connection instantly:

- **The Restaurant Address (Public Gateway URL):** `https://172.208.96` *(This is your live Azure VM IP address door).*
- **The Internal Kitchen Seat:** `http://127.0.0.1:8000` *(Where Flask sits inside the Linux system).*
- **The Action (HTTP Method):** `POST` *(This explicitly translates to "Write a brand new entry").*
- **The Package Style (Content-Type):** `application/json` *(A clean, standardized format for computer text blocks).*
- **The Custom Secret Password Stamp (Authentication Header):** `X-Matecca-API-Key: MateccaSecretLiveToken2026_Secure`

The Standard Form Envelope Layout:

json

```
{
  "name": "Jane Doe",
  "email": "jane@acmelabs.com"
}
```

Use code with caution.

What the Waiter Hands Back to WordPress:

- **HTTP 200 OK (Success!):** "I processed the domain, figured out they work at AcmeLabs, graded them a score of 85, and locked it in the database vault."
- **HTTP 401 Unauthorized (Bouncer Block):** "The secret password header stamp is missing or wrong. Access denied."

- **HTTP 400 Bad Request (Missing Items):** "You forgot to fill out the name or email slot in the data envelope."

---

3. Hardened Production Application Source Code (`main.py`)

This secure source code is deployed inside your remote cloud application workspace path at `/home/azureuser/python_api/main.py`. It has been completely optimized to ensure it doesn't throw indentation bugs or connection drops.

python

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import mysql.connector

# Initialize our web server framework
app = Flask(__name__)

# Enforce Cross-Origin Resource Sharing (CORS) rules natively
CORS(app)

# Enterprise Token Perimeter Definition (The master bouncer key)
MATECCA_SECRET_TOKEN = "MateccaSecretLiveToken2026_Secure"

def get_db_connection():
    # Establishes connection frameworks to production database schemas
    # Bound safely internally inside the VM network loop (localhost /
127.0.0.1)
    return mysql.connector.connect(
        host="localhost",
        user="crm_user",
        password="Matecca_Secure_2026!",  # Hardened custom database
credential
        database="saas_crm"
    )

@app.route('/api/v1/leads', methods=['POST'])
def create_lead():
    # STEP 1: Enforce strict token checks before parsing the incoming
request body
    client_api_key = request.headers.get("X-Matecca-API-Key")
    if client_api_key != MATECCA_SECRET_TOKEN:
```

```
        return jsonify({"status": "error", "message": "Unauthorized API
Access Blocked"}), 401

    # STEP 2: Extract inputs cleanly out of the secure JSON request body
(NOT public URL strings)
    request_data = request.get_json()
    if not request_data:
        return jsonify({"status": "error", "message": "Invalid or missing
payload body"}), 400

    name = request_data.get('name')
    email = request_data.get('email')

    if not name or not email:
        return jsonify({"status": "error", "message": "Required validation
values missing"}), 400

    # STEP 3: Lead Enrichment Core Pipeline Operations
    # Example input: 'testy@acmelabs.com'
    domain = email.split('@')[-1]                  # Snipps string at '@'
to get 'acmelabs.com'
    company_name = domain.split('.')[0].capitalize() # Snipps '.com' and
capitalizes it to 'Acmelabs'

    # STEP 4: Automated Evaluation Engine Scoring Rules
    if domain in ["gmail.com", "yahoo.com", "hotmail.com"]:
        industry = "Unknown (Personal Email Address)"
        lead_score = 10 # Low-tier personal lead
    else:
        industry = "B2B Tech Sector"
        lead_score = 85 # Premium high-value corporate lead!

    # STEP 5: Cloud Database Persistence Engine (The Vault Lockup)
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # We use secure '%s' text shields to completely block malicious
SQL Injection hacker attacks
        sql = "INSERT INTO crm_leads (name, email, company, industry,
lead_score) VALUES (%s, %s, %s, %s, %s)"
        values = (name, email, company_name, industry, lead_score)

        cursor.execute(sql, values)
        conn.commit() # Slam the vault shut and lock it
```

```
    except mysql.connector.Error as err:
        # Prevent dangerous system stack traces from leaking to the public
frontend window
        return jsonify({"status": "error", "message": "Internal
persistence engine conflict"}), 500
    finally:
        # Clean up and wash the plates after the request is finished
        if 'cursor' in locals(): cursor.close()
        if 'conn' in locals(): conn.close()

    # STEP 6: Return the happy success receipt to your WordPress screen
    return jsonify({
        "status": "success",
        "message": f"Lead profile processed and synchronized for
{company_name}",
        "score": lead_score,
        "company": company_name,
        "industry": industry
    }), 200

if __name__ == '__main__':
    # Force the local listener loop loop; Nginx handles secure public
HTTPS proxying on Port 443
    app.run(host='127.0.0.1', port=8000)
```

Use code with caution.

---

4. Line-by-Line Code Breakdowns (Plain English)

Section 1: Fetching Our Construction Tools (Lines 1–3)

- **Line 1 (`from flask import Flask, request, jsonify`):** Loads the code framework. `Flask` sets up the layout blueprint, `request` opens up the waiter's pockets to look for envelopes, and `jsonify` formats plain Python data into neat digital receipt packages.
- **Line 2 (`from flask_cors import CORS`):** Imports browser permission exceptions, ensuring your outside WordPress local environment is safely allowed to push traffic down to your Azure box.
- **Line 3 (`import mysql.connector`):** Hooks up the native database driver wire so

Python can type data straight into your SQL files.

Section 2: Building the Restaurant Foundation (Lines 5–10)

- **Line 5 (`app = Flask(__name__)`):** Turns the initialization key to convert a standard static text file into a living, breathing web server listener.
- **Line 6 (`CORS(app)`):** Lowers the perimeter gate to allow communication from your local development machine.
- **Line 10 (`MATECCA_SECRET_TOKEN = "..."`):** Creates the master password card that the waiter uses to verify all incoming packages.

Section 3: Connecting to the Vault (Lines 12–19)

- **Lines 12–19 (`def get_db_connection():`):** Builds a quick-access shortcut tool to talk to your database. Because the host parameter is locked directly to `localhost`, the database is completely walled off from the outside internet.

Section 4: Validation & Payload Parsing (Lines 21–36)

- **Line 21 (`@app.route('/api/v1/leads', methods=['POST'])`):** Directs the waiter to stand at one specific room entrance and process requests using only the `POST` saving format.
- **Lines 23–25:** The waiter scans the header stamps. If the password key doesn't match perfectly, they dump the envelope and send back a `401 Unauthorized` block.
- **Line 28 (`request.get_json()`):** Unwraps the delivery box cleanly to read the items. If the customer forgot a name or email, it throws an early `400 Bad Request` warning.

Section 5: Enrichment Automation Engine (Lines 38–47)

- **Line 39 (`email.split('@')[-1]`):** Works exactly like scissors. It trims the account front loop and leaves just the core domain extension (`acmelabs.com`).
- **Line 40 (`company_name = ...`):** Cuts off the trailing `.com` and capitalizes the word so it reads nicely (turning `acmelabs` into `Acmelabs`).
- **Lines 42–47:** Runs an automated valuation check. Corporate users get premium sector flags and high scores, while basic consumer addresses (`gmail.com`) get minimum ranks.

Section 6: Guarded Transaction Layer & Persistence (Lines 49–61)

- **Line 49 (`try:`):** An internal safety net envelope. If the database crashes or hits a port conflict, the error is isolated quietly so the script doesn't freeze or lock up.
- **Lines 50–55 (`cursor.execute(sql, values)`):** Pushes your clean variables into your SQL columns, utilizing secure data array parameters to neutralize hacker injection scripts.
- **Lines 58–61 (`finally:`):** The final clean-sweep broom. This forces the open database traffic valves to snap shut securely so they never waste your cloud server's RAM memory loops.

---

5. Enterprise Process & Service Management (The Automated Manager)

To stop your API from turning off whenever you log out of your terminal, your script is provisioned as an active background **Linux Systemd Service**. Think of this like hiring a tireless manager to watch your script around the clock.

To configure your automated manager file, execute this inside your VM terminal:

bash

```
sudo nano /etc/systemd/system/flask-crm.service
```

Use code with caution.

Drop this configuration data block straight inside that window and save:

ini

```
[Unit]
Description=Matecca Industries Production Flask REST API Daemon Service
After=network.target mysql.service

[Service]
User=azureuser
WorkingDirectory=/home/azureuser/python_api
# Points directly to your isolated Python virtual environment binary package
ExecStart=/home/azureuser/python_api/venv/bin/python
```

```
/home/azureuser/python_api/main.py
# If the code ever crashes, the manager restarts it instantly in 5
seconds!
Restart=always
RestartSec=5
# Save all operational events and error history loops into a tracker file
StandardOutput=append:/home/azureuser/python_api/flask.log
StandardError=append:/home/azureuser/python_api/flask.log

[Install]
WantedBy=multi-user.target
```

Use code with caution.

Essential System Unit Process Commands:

- `sudo systemctl daemon-reload` ➔ Tells Linux you updated the manager's background rule sheet.
- `sudo systemctl enable flask-crm.service` ➔ Forces the API to boot automatically whenever the Azure machine turns on.
- `sudo systemctl start flask-crm.service` ➔ Launches the background runner engine right now.
- `sudo systemctl status flask-crm.service` ➔ Asks the manager if the API is running perfectly. Look for a bright green status tag.
- `tail -f /home/azureuser/python_api/flask.log` ➔ Opens the script's live tracking logging window on your screen to watch incoming form leads stream past in real time.

---

6. Comprehensive Production Troubleshooting Guide

❌ Error Profile 1: HTTP 401 Unauthorized Block

- **The Waiter Cause:** Your local WordPress site is knocking on the cloud door, but it either forgot its secret key stamp or used an outdated signature string.
- **The Fix:** Open your local `matecca-crm-gateway.php` file on your PC and confirm that the `'X-Matecca-API-Key'` text matches your cloud `MATECCA_SECRET_TOKEN` exactly. Ensure you added `underscores_in_headers on;` inside your Azure

Nginx profile script, otherwise Nginx silently strips the header out.

❌ Error Profile 2: HTTP 400 Bad Request / "Invalid payload body"

- **The Waiter Cause:** Your data envelope safely bypassed the bouncer, but when the waiter looked inside the package, the form parameters were blank or scrambled.
- **The Fix:** Check your local site page HTML structure. Make sure your form wrapper uses `id="native-lead-form"` and your data elements use `id="lead-name"` and `id="lead-email"`. If these IDs don't match, JavaScript cannot collect the values.

❌ Error Profile 3: Database Connection Refused (InterfaceError 2003)

- **The Waiter Cause:** The waiter walked back to the kitchen to drop off your lead, but the kitchen doors were locked tight because the database software crashed or stopped.
- **The Fix:** SSH into your Azure console and query: `sudo systemctl status mysql`. If the service is red or dead, revive it instantly by executing: `sudo systemctl restart mysql`.

❌ Error Profile 4: Front-End "CORS Network Blocked"

- **The Waiter Cause:** Your web browser blocked your local environment page from reading the response because it assumes an unauthorized outside system is trying to sneak files in.
- **The Fix:** Open your cloud `main.py` script and verify that `from flask_cors import CORS` and `CORS(app)` are explicitly written right underneath your application setup configuration line.

❌ Error Profile 5: Background Manager Failure (Code 1/FAILURE)

- **The Waiter Cause:** The automated system manager tried to start your API, but it tripped because it couldn't locate where your Python executable tool wrapper is saved on the hard drive.
- **The Fix:** Run `sudo nano /etc/systemd/system/flask-crm.service` and verify your `ExecStart` pathway points explicitly to your isolated project virtual folder binary layout: `/home/azureuser/python_api/venv/bin/python`.